

Лабораторная работа №4

Реализация основных моделей в агентном подходе

Разанацуа Сара Естэлл

Содержание

1	Цель работы	5
2	Задание	6
3	Выполнение лабораторной работы	7
3.1	Эпидемиологическая модели SIR	7
3.2	Базовый эксперимент	8
3.3	Сканирование коэффициента заразности	10
3.4	Исследование эффекта миграции	12
3.5	Многокритериальная оптимизация параметров	14
3.6	Сводная визуализация результатов	15
4	Выводы	18
	Список литературы	19

Список иллюстраций

3.1	Рис		7
3.2	Рис		7
3.3	Рис		8
3.4	Рис		9
3.5	Рис		10
3.6	Рис		11
3.7	Рис		11
3.8	Рис		12
3.9	Рис		13
3.10	Рис		13
3.11	Рис		14
3.12	Рис		15
3.13	Рис		16
3.14	Рис		17

Список таблиц

1 Цель работы

Создадим агентную модель распространения инфекционного заболевания на основе классической компартментальной модели SIR (Susceptible-Infectious-Recovered). Модель будет реализована с использованием пакета Agents.jl. В отличие от классической модели на дифференциальных уравнениях, агентный подход позволит учесть индивидуальные характеристики, пространственную структуру и стохастичность процессов.

2 Задание

1. Создать рабочий каталог для кода.
2. Установить необходимые пакеты.
3. Выполнить предложенный код.
4. Преобразовать код в литературный стиль.
5. Сгенерировать из литературного кода:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.
6. Выполнить код из jupyter notebook.
7. Интегрировать документацию в формате Quarto в отчёт.
8. Добавить в код в литературном стиле вычисление для набора параметров.
9. Сгенерировать из литературного кода с параметрами:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.
10. Выполнить код из jupyter notebook с параметрами.
11. Интегрировать документацию с параметрами в формате Quarto в отчёт.
 - Результирующие файлы не удаляйте, выложите на git.

3 Выполнение лабораторной работы

3.1 Эпидемиологическая модели SIR

- Мы создадим файл `src/sir_model.jl` и запустим его. (рис. 3.1).



```

# Определение типа агента
@agent struct Person{GraphAgent}
    days_infected::Int # количество дней с момента заражения
    status::Symbol # :S, :I, :R
end

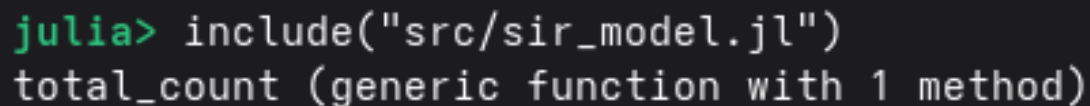
# Функция инициализации модели
function initialize_sir()
    Ns = [1000, 1000, 1000], # численность по 1000 городам
    migration_rates = nothing, # матрица миграции
    beta_uninfected = [0.5, 0.5, 0.5], # передача невыявленными
    beta_infected = [0.05, 0.05, 0.05], # передача выявленными
    infection_period = 14,
    detection_time = 7,
    death_rate = 0.02,
    reinfection_probability = 0.1,
    Is = [0, 0, 1], # начальное количество инфицированных
    seed = 42,
    n_steps = 100,
end

rng = Xoshiro(seed)
C = length(Ns) # количество городов

# Если матрица миграции не задана, создаём реалистичную
if migration_rates == nothing
    migration_rates = zeros{C, C}
    for i = 1:C
        for j = 1:C
            # Вероятность миграции пропорциональна размеру целевого города
            migration_rates[i, j] = (Ns[i] * Ns[j]) / Ns[i]
        end
    end
end
```

Рис. 3.1: Рис

- Вывод в консоль. (рис. 3.2).



```

julia> include("src/sir_model.jl")
total_count (generic function with 1 method)
```

Рис. 3.2: Рис

3.2 Базовый эксперимент

- Запускаем один эксперимент с фиксированными параметрами (по умолчанию) и сохраняем динамику численности агентов. Это служит для проверки работоспособности модели и получения базового понимания эпидемического процесса. (рис. 3.3).



```
# Параметры эксперимента
params = Dict{
    :N0 => [1000, 1000, 1000],
    :p_und => [0.5, 0.5, 0.5],
    :p_det => [0.05, 0.05, 0.05],
    :infection_period => 14,
    :detection_time => 7,
    :death_rate => 0.02,
    :reinfection_probability => 0.1,
    :is => [0, 0, 1],
    :seed => 42,
    :n_steps => 100,
}

# Инициализация модели
model = initialize_sir(; params...)

# Подготовка массивов для хранения данных
times = Int[]
S_vals = Int[]
I_vals = Int[]
R_vals = Int[]
total_vals = Int[]

# Запуск симуляции вручную
for step = 1:params[:n_steps]
    Agents.step!(model, 1)

    push!(times, step)
    push!(S_vals, susceptible_count(model))
    push!(I_vals, infected_count(model))
    push!(R_vals, recovered_count(model))
    push!(total_vals, total_count(model))
end
```

Рис. 3.3: Рис

- Мы создадим файл `src/sir_run_basic.jl` и запустим его. (рис. 3.4).

```

julia> touch("scripts/sir_scan_beta.jl")
"scripts/sir_scan_beta.jl"

julia> include("scripts/sir_scan_beta.jl")
Завершён эксперимент с beta = 0.1, seed = 42
Завершён эксперимент с beta = 0.1, seed = 43
Завершён эксперимент с beta = 0.1, seed = 44
Завершён эксперимент с beta = 0.2, seed = 42
Завершён эксперимент с beta = 0.2, seed = 43
Завершён эксперимент с beta = 0.2, seed = 44
Завершён эксперимент с beta = 0.3, seed = 42
Завершён эксперимент с beta = 0.3, seed = 43
Завершён эксперимент с beta = 0.3, seed = 44
Завершён эксперимент с beta = 0.4, seed = 42
Завершён эксперимент с beta = 0.4, seed = 43
Завершён эксперимент с beta = 0.4, seed = 44
Завершён эксперимент с beta = 0.5, seed = 42
Завершён эксперимент с beta = 0.5, seed = 43
Завершён эксперимент с beta = 0.5, seed = 44
Завершён эксперимент с beta = 0.6, seed = 42
Завершён эксперимент с beta = 0.6, seed = 43
Завершён эксперимент с beta = 0.6, seed = 44
Завершён эксперимент с beta = 0.7, seed = 42
Завершён эксперимент с beta = 0.7, seed = 43
Завершён эксперимент с beta = 0.7, seed = 44
Завершён эксперимент с beta = 0.8, seed = 42
Завершён эксперимент с beta = 0.8, seed = 43
Завершён эксперимент с beta = 0.8, seed = 44
Завершён эксперимент с beta = 0.9, seed = 42
Завершён эксперимент с beta = 0.9, seed = 43
Завершён эксперимент с beta = 0.9, seed = 44
Завершён эксперимент с beta = 1.0, seed = 42
Завершён эксперимент с beta = 1.0, seed = 43
Завершён эксперимент с beta = 1.0, seed = 44
Результаты сохранены в data/beta_scan_all.csv и plots/beta_scan.png

```

Рис. 3.4: Рис

- График: plots/sir_basic_dynamics.png — четыре линии: $S(t)$, $I(t)$, $R(t)$ и общая численность. Позволяет визуально оценить пик эпидемии, скорость распространения и влияние смертности. (рис. 3.5).

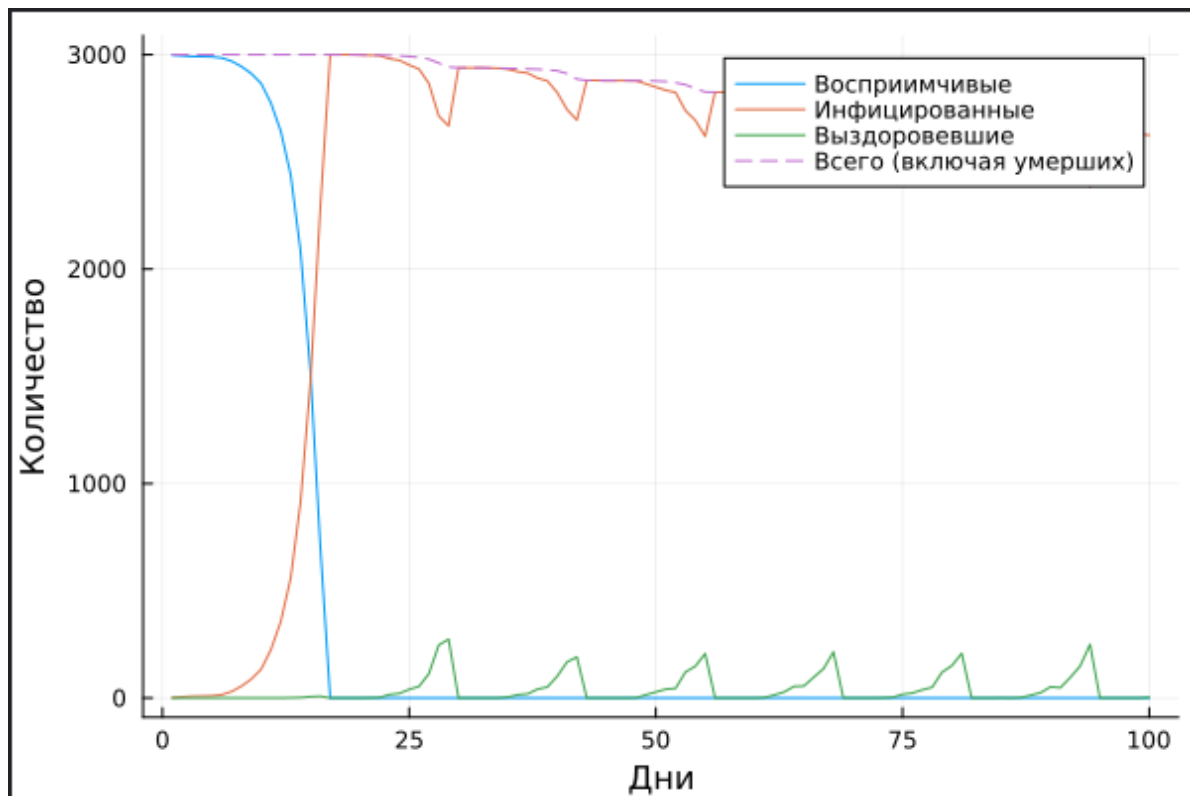


Рис. 3.5: Рис

3.3 Сканирование коэффициента заразности

- Исследуем, как изменение базовой заразности (β_{und} и пропорционально β_{det}) влияет на эпидемические показатели: пик заболеваемости, долю переболевших, число умерших. Выполняется параметрическое сканирование с несколькими повторными прогонами для учёта стохастичности. (рис. 3.6).

```

# Функция для запуска одного эксперимента и возврата метрик
function run_experiment(p)
    # Создаём  $\beta_{und}$  и  $\beta_{det}$  на основе скалярного  $\beta$ 
    beta = p[:beta]
     $\beta_{und}$  = fill(beta, 3)
     $\beta_{det}$  = fill(beta/10, 3)
    # Передаём в модель
    model = initialize_sir(;
        Ns = p[:Ns],
         $\beta_{und}$  =  $\beta_{und}$ ,
         $\beta_{det}$  =  $\beta_{det}$ ,
        infection_period = p[:infection_period],
        detection_time = p[:detection_time],
        death_rate = p[:death_rate],
        reinfection_probability = p[:reinfection_probability],
        Is = p[:Is],
        seed = p[:seed],
        n_steps = p[:n_steps], # этот параметр не используется в initialize_sir, но может быть нужен для цикла
    )

    infected_fraction(model) =
        count(a.status == :i for a in allagents(model)) / nagents(model)
    peak_infected = 0.0

    for step = 1:p[:n_steps]
        # Ручной шаг (безопасный)
        agent_ids = collect(allids(model))

        for id in agent_ids
            agent = try
                model[id]
            catch
                nothing
            end
        end
    end
end

```

Рис. 3.6: Рис

- Создание файла scripts/sir_scan_beta.jl и запустим его. (рис. 3.7).

```

julia> touch("scripts/sir_migration_effect.jl")
"scripts/sir_migration_effect.jl"

julia> include("scripts/sir_migration_effect.jl")
Завершён эксперимент с migration_intensity = 0.0, seed = 42
Завершён эксперимент с migration_intensity = 0.0, seed = 43
Завершён эксперимент с migration_intensity = 0.0, seed = 44
Завершён эксперимент с migration_intensity = 0.1, seed = 42
Завершён эксперимент с migration_intensity = 0.1, seed = 43
Завершён эксперимент с migration_intensity = 0.1, seed = 44
Завершён эксперимент с migration_intensity = 0.2, seed = 42
Завершён эксперимент с migration_intensity = 0.2, seed = 43
Завершён эксперимент с migration_intensity = 0.2, seed = 44
Завершён эксперимент с migration_intensity = 0.3, seed = 42
Завершён эксперимент с migration_intensity = 0.3, seed = 43
Завершён эксперимент с migration_intensity = 0.3, seed = 44
Завершён эксперимент с migration_intensity = 0.4, seed = 42
Завершён эксперимент с migration_intensity = 0.4, seed = 43
Завершён эксперимент с migration_intensity = 0.4, seed = 44
Завершён эксперимент с migration_intensity = 0.5, seed = 42
Завершён эксперимент с migration_intensity = 0.5, seed = 43
Завершён эксперимент с migration_intensity = 0.5, seed = 44
Результаты сохранены в data/migration_scan_all.csv и plots/migration_effect.png

```

Рис. 3.7: Рис

- График позволяет найти пороговое значение β , при котором возникает

эпидемия, и оценить, как увеличивается нагрузка на систему с ростом заразности (рис. 3.8).

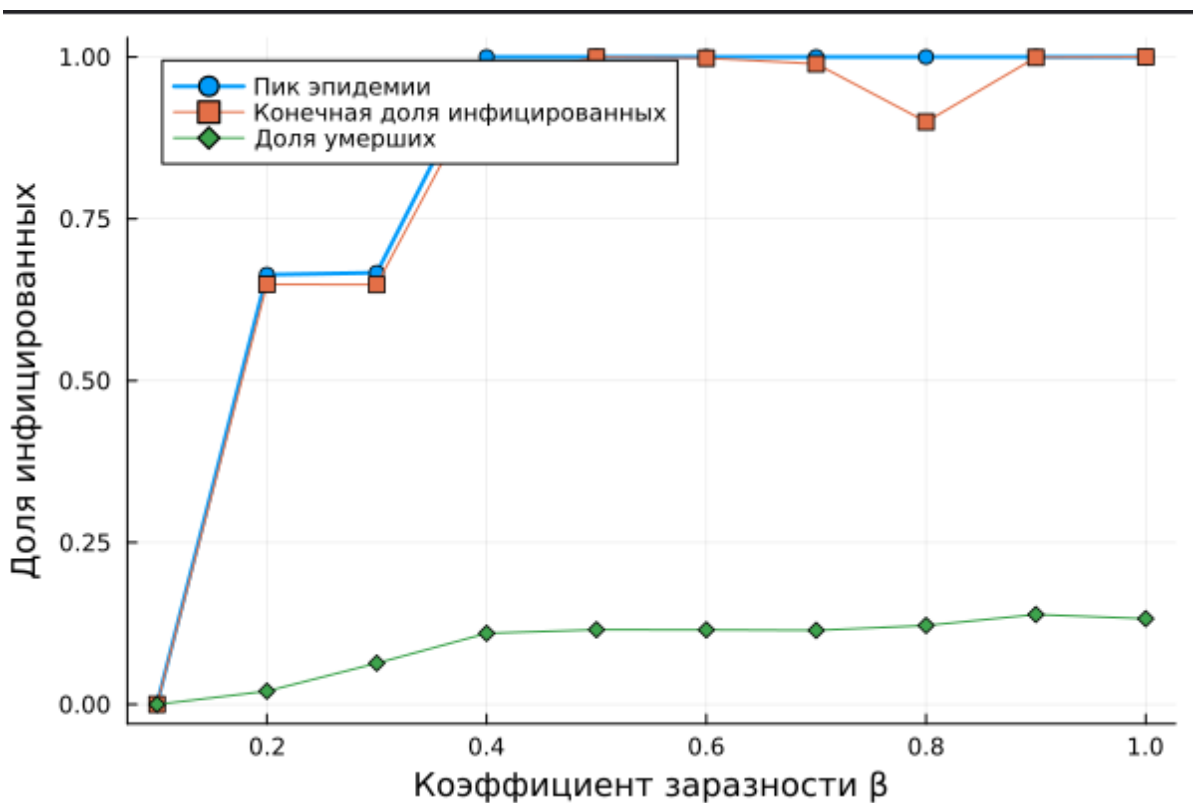


Рис. 3.8: Рис

3.4 Исследование эффекта миграции

- Исследует, как интенсивность перемещения людей между городами влияет на скорость распространения эпидемии (время достижения пика) и масштаб пика. Инфекция начинается только в одном городе, остальные изначально здоровы. (рис. 3.9).

```

sir_model.jl | sir_run_basic.jl | sir_scan_beta.jl | sir_migration_effect.jl | sir_optimize_parameters.jl
~/work/study/2025-2026/simulation-modeling/study_2025-2026/simulation-modeling/lab04/project/scripts

# Функция для создания матрицы миграции с заданной интенсивностью
function create_migration_matrix(C, intensity)
    M = ones(C, C) .* intensity ./ (C-1)
    for i = 1:C
        M[i, i] = 1 - intensity
    end
    return M
end

# Функция для измерения времени достижения пика
function peak_time(n)
    # Создаём матрицу миграции на основе интенсивности
    migration_rates = create_migration_matrix(p[:C], p[:migration_intensity])
    model = initialize_sir(;
        Ns = p[:Ns],
        beta_und = p[:beta_und],
        beta_det = p[:beta_det],
        infection_period = p[:infection_period],
        detection_time = p[:detection_time],
        death_rate = p[:death_rate],
        reinfection_probability = p[:reinfection_probability],
        Is = p[:Is],
        seed = p[:seed],
        migration_rates = migration_rates,
    )

    infected_frac(model) = count(a.status == :I for a in allagents(model)) / nagents(model)
    peak = 0.0
    peak_step = 0

    for step = 1:p[:n_steps]
        # Единица war
        agent_ids = collect(allids(model))
        for id in agent_ids
            agent = try
                model[id]
            catch
            end
        end
    end
end

```

Рис. 3.9: Рис

- График демонстрирует, как ускорение обмена людьми между городами приводит к более раннему и более высокому пику эпидемии (рис. 3.10).

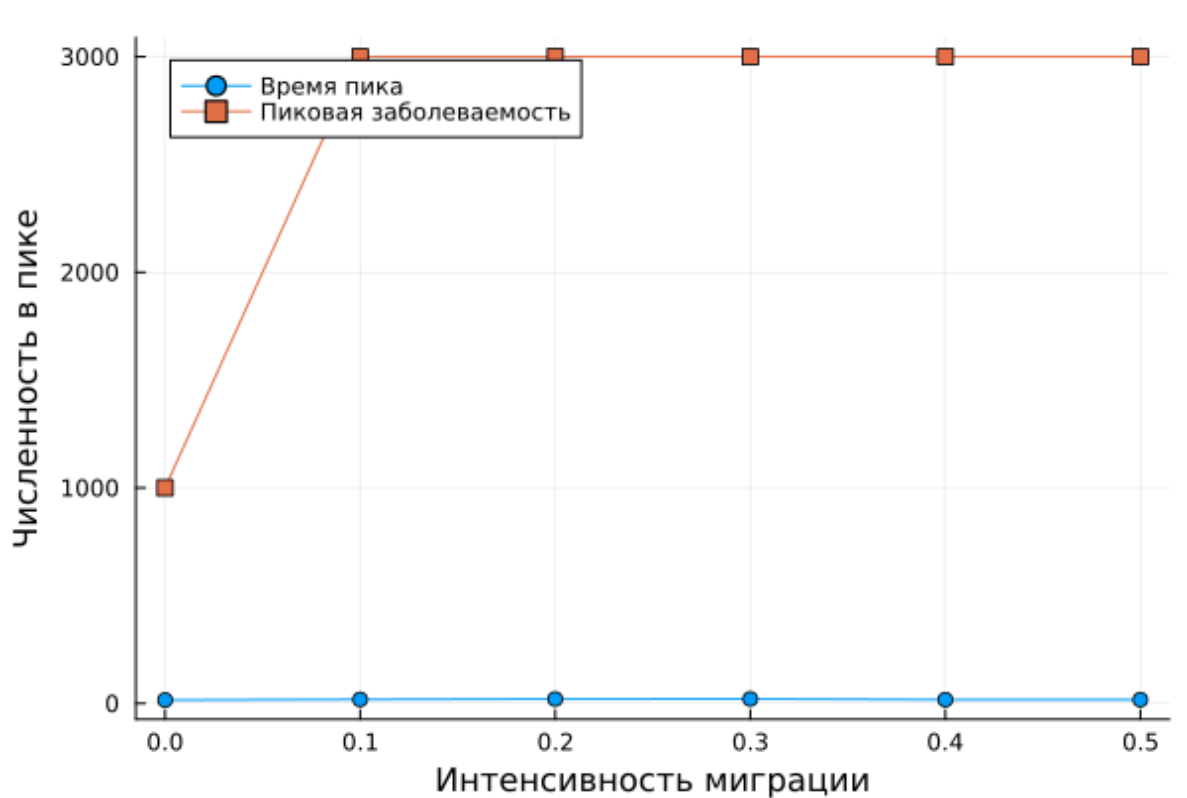


Рис. 3.10: Рис

3.5 Многокритериальная оптимизация параметров

- Ищем оптимальные комбинации параметров, минимизирующие одновременно два критерия: пиковую заболеваемость и долю умерших. Используем эволюционный алгоритм (Borg MOEA) из пакета BlackBoxOptim. (рис. 3.11).



```
#!/usr/bin/env julia
# sir_optimize_parameters.jl
# Целевая функция: минимизируем пиковую заболеваемость и смертность

function cost_multi(x)
    # x[1]: beta, x[2]: death_rate, x[3]: detection_time
    model = initialize_sir(
        Ns = [1000, 1000, 1000],
        beta = fill(x[1], 3),
        beta_det = fill(x[1]/10, 3),
        infection_period = 14,
        detection_time = round(Int, x[2]),
        death_rate = x[3],
        reinfection_probability = 0.1,
        Is = [0, 0, 1],
        seed = 42,
        n_steps = 100,
    )

    infected_frac(model) = count(a.status == :I for a in allagents(model)) / nagents(model)
    dead_count(model) = 3000 - nagents(model)

    peak_infected = 0.0

    # Запускаем с несколькими повторами
    replicates = 5
    peak_vals = Float64[]
    dead_vals = Int[]

    for rep = 1:replicates
        model = initialize_sir(
            Ns = [1000, 1000, 1000],
            beta = fill(x[1], 3),
            beta_det = fill(x[1]/10, 3),
            infection_period = 14,
            detection_time = round(Int, x[2]),
            death_rate = x[3],
            reinfection_probability = 0.1,
            Is = [0, 0, 1],
            seed = 42 + rep,
        )
```

Рис. 3.11: Рис

- Создание файла scripts/sir_optimize_parameters.jl и запустим его. (рис. 3.12).

```
Optimization stopped after 1 steps and 889.28 seconds
Termination reason: Max time (120.0 s) reached
Steps per second = 0.00
Function evals per second = 0.06
Improvements/step = NaN
Total function evaluations = 51
```

```
Best candidate found: [0.122956, 5.34903, 0.0237749]
```

```
Fitness: (0.00067, 0.00000) agg=0.00067
```

```
Оптимальные параметры:
```

```
 $\beta_{und} = 0.12295579898158256$ 
```

```
Время выявления = 5 дней
```

```
Смертность = 0.023774934501594132
```

```
Достигнутые показатели:
```

```
Пик заболеваемости: 0.00066666666666666666
```

```
Доля умерших: 0.0
```

Рис. 3.12: Рис

- Вывод в консоль: оптимальные параметры и соответствующие им значения критериев.

3.6 Сводная визуализация результатов

- Этот скрипт загружает результаты параметрического сканирования (файл data/beta_scan_all.csv, созданный scan_beta.jl) и строит единый составной график, объединяющий три панели:

— пик эпидемии и конечная доля инфицированных; — число умерших; — доля выздоровевших.

График сохраняется как `plots/comprehensive_analysis.png`. Скрипт предназначен для итогового отчёта и не требует повторного запуска симуляций. (рис. 3.13).



```
using DrWatson
@quickactivate "project1"
using Agents, DataFrames, Plots, CSV

include(sourcedir("sir_model.jl"))

# Загружаем результаты сканирования
df = CSV.read(datadir("beta_scan_all.csv"), DataFrame)

# Создаём составной график
p1 = plot(df.beta, df.peak, label = "Пик", xlabel = "β", ylabel = "Доля инфицированных")
plot!(p1, df.beta, df.final_inf, label = "Конечная")
p2 = plot(df.beta, df.deaths, xlabel = "β", ylabel = "Число умерших")
p3 = plot(df.beta, df.final_rec, xlabel = "β", ylabel = "Доля выздоровевших")
plot(p1, p2, p3, layout = (3, 1), size = (800, 900))
```

Рис. 3.13: Рис

- График: `plots/comprehensive_analysis.png` (рис. 4.4). Три панели, позволяющие одновременно оценить:

— порог возникновения эпидемии (рост пика); — нелинейное увеличение смертности; — насыщение доли переболевших.

- В консоль может выводиться дополнительная статистика (например, значения β , при которых пик превышает 5%). (рис. 3.14).

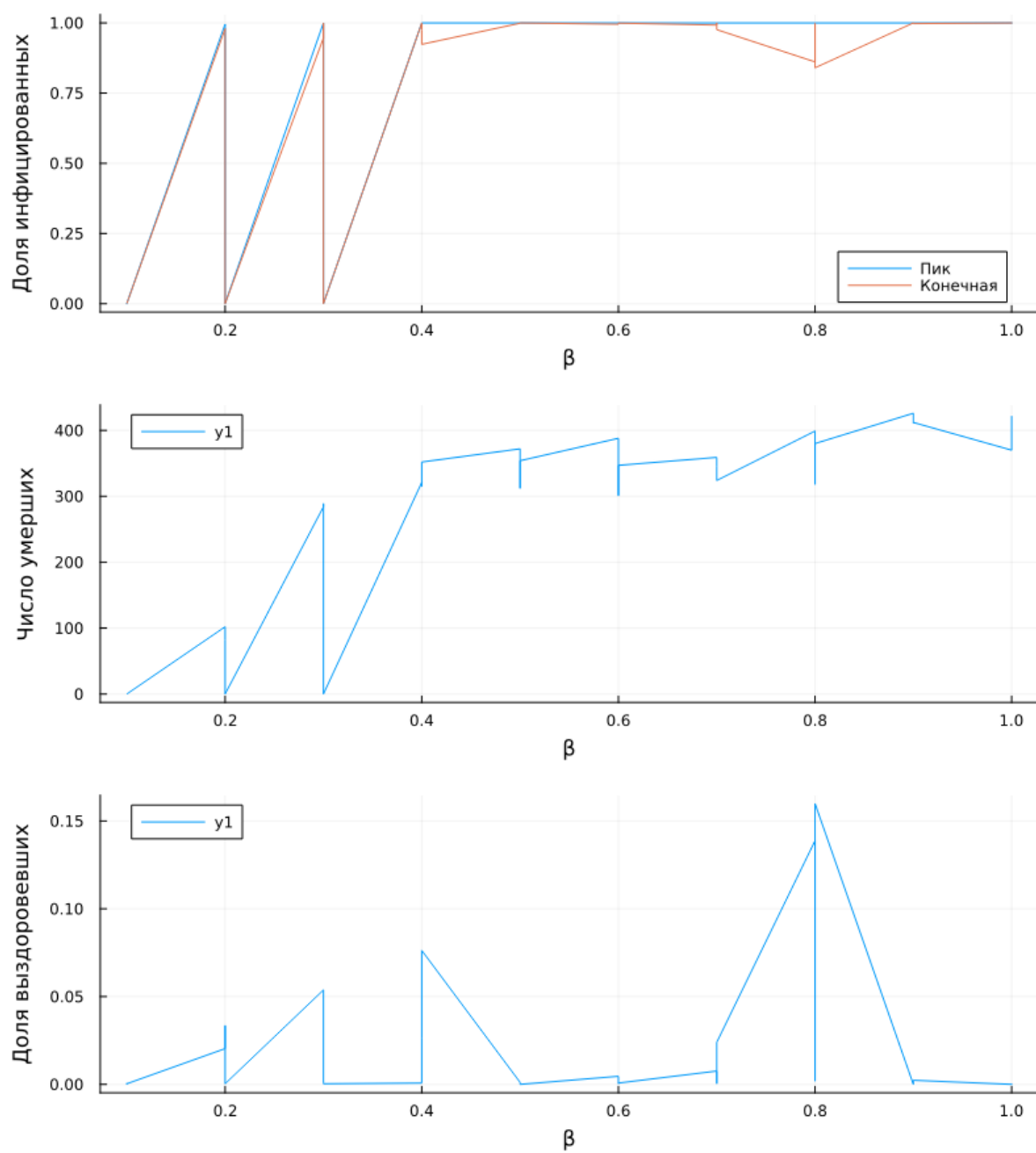


Рис. 3.14: Рис

4 Выводы

- Я выполнила лабораторную работу.

Список литературы